

ARRAYS AND SEARCHING

Lesson Objectives

By the end of this lesson, students should be able to:

- Understand what an **array** is and how to represent it in pseudocode.
 - Explain and implement **linear search** and **binary search** algorithms.
 - Write pseudocode to search for a student's record in an array.
 - Perform a simple **Git branching task** for version control.
-

1. Introduction to Arrays

An **array** is a **collection of elements of the same type**, stored in a sequence under one name. Each element is identified by an **index (or position)**.

Array Characteristics

- Stores multiple items of the same data type.
 - Accessed using an **index** (starting from 1 or 0 depending on the language).
 - Has a **fixed size** (number of elements).
-

Pseudocode Declaration Examples

```
DECLARE scores[5] AS INTEGER
DECLARE names[10] AS STRING
```

Assigning and Accessing Values

```
scores[1] = 85
scores[2] = 90
PRINT scores[2]    // Output: 90
```

Example: Sum of Array Elements

```
DECLARE arr[5] AS INTEGER
sum = 0

FOR i = 1 TO 5
    INPUT arr[i]
    sum = sum + arr[i]
NEXT i

PRINT "Sum = ", sum
```

2. Searching in Arrays

Searching means **finding a specific element** in a collection.

There are two common searching methods:

1. **Linear Search** (Sequential Search)
2. **Binary Search**

◆ A. Linear Search

Linear search checks each element **one by one** until the target is found.

Algorithm Steps

1. Start from the first element.
 2. Compare each element with the target value.
 3. If a match is found, report its position.
 4. If the end of the list is reached, report “not found.”
-

Pseudocode – Linear Search

```
PROCEDURE LinearSearch(array, n, target)
    FOUND = FALSE
```

```
FOR i = 1 TO n
    IF array[i] = target THEN
        PRINT "Item found at position ", i
        FOUND = TRUE
        EXIT FOR
    ENDIF
NEXT i

IF FOUND = FALSE THEN
    PRINT "Item not found"
ENDIF
ENDPROCEDURE
```

Example:

```
array = [12, 25, 37, 49, 55]
target = 37
```

Output:

Item found at position 3

◆ B. Binary Search (Introduction)

Binary search is **faster** but only works on **sorted arrays**.

Algorithm Steps

1. Find the middle element.
 2. If it matches the target → found.
 3. If target < middle → search the left half.
 4. If target > middle → search the right half.
 5. Repeat until found or array is exhausted.
-

Pseudocode – Binary Search

```

PROCEDURE BinarySearch(array, n, target)
    low = 1
    high = n
    FOUND = FALSE

    WHILE low <= high AND FOUND = FALSE DO
        mid = (low + high) DIV 2
        IF array[mid] = target THEN
            PRINT "Item found at position ", mid
            FOUND = TRUE
        ELSE IF array[mid] < target THEN
            low = mid + 1
        ELSE
            high = mid - 1
        ENDIF
    ENDWHILE

    IF FOUND = FALSE THEN
        PRINT "Item not found"
    ENDIF
ENDPROCEDURE

```

⚡ Comparison: Linear vs Binary Search

Feature	Linear Search	Binary Search
Works on sorted data?	No	Yes
Speed	Slower	Faster
Approach	Sequential	Divide and Conquer
Complexity	O(n)	O(log n)

3. Activity – Searching Student Records

Problem

Given an array of student names, write pseudocode to **search for a particular student** by name using **Linear Search**.

Pseudocode

```

DECLARE students[5] AS STRING
DECLARE name AS STRING
FOUND = FALSE

FOR i = 1 TO 5
    INPUT students[i]
NEXT i

INPUT name

FOR i = 1 TO 5
    IF students[i] = name THEN
        PRINT name, " found at position ", i
        FOUND = TRUE
        EXIT FOR
    ENDIF
NEXT i

IF FOUND = FALSE THEN
    PRINT "Student not found"
ENDIF

```

Example Input/Output

```

students = ["Joy", "Amos", "Lina", "Ben", "Ruth"]
name = "Lina"

```

Output: Lina found at position 3

4. Git Task – Version Control

You'll now integrate your algorithm practice with **Git** for version management.

Task Instructions

1. Open your course repository folder.
2. Create a **new branch** for your Week 5 solutions.

Command Line Steps:

```
git checkout -b week5-arrays
```

3. Save your pseudocode files in this branch.
4. Add and commit your changes:

```
git add .  
git commit -m "Added Week 5 array and search pseudocode"
```

5. Push the branch to your GitHub repository:

```
git push -u origin week5-arrays
```

✓ *This task teaches collaboration and safe code updates without affecting the main branch.*

5. Exercises

1. Write pseudocode to count how many times a value appears in an array.
 2. Modify the linear search pseudocode to return the **index** of the first match.
 3. Create pseudocode to find the **largest value** in an integer array.
 4. Explain why binary search cannot be used on unsorted data.
 5. Write pseudocode for binary search on an array of **student ID numbers**.
-